

Rapport : Concevoir des solutions répondant à des besoins et des problèmes complexes

Installer une solution dans son écosystème et dans le temps

12 janvier 2026

Membres du groupe

Florestan BOUCHARLAT
Steven DELALAIN
Christophe LUU
Daniel MALINOWSKI
Thomas SZABO



Sommaire

1. Introduction générale.....	1
2. Méthodologie et cadre expérimental.....	1
2.1 Principe général de simulation.....	1
2.2 Démarche de benchmarking et reproductibilité.....	2
2.3 Métriques observées.....	2
3. Étude de cas n°1 : Waterfall.....	3
3.1 Description du système.....	3
3.2 Paramètres du modèle.....	4
3.3 Comportement du système et stabilité.....	5
3.4 Évaluation par métriques.....	6
3.5 Synthèse et recommandations.....	9
4. Étude de cas n°2 : Channels and Dams.....	9
4.1 Description du système.....	9
4.2 Paramètres du modèle.....	10
4.3 Comportement du système et stabilité.....	11
4.4 Évaluation du système au regard des métriques.....	12
4.5 Synthèse et recommandations.....	15
5. Discussion transversale et limites.....	16
6. Conclusion et recommandations générales.....	17
Annexes.....	18

1. Introduction générale

Contexte des systèmes d'attente

Les systèmes d'attente occupent une place centrale dans de nombreux systèmes informatiques modernes, qu'il s'agisse de la gestion de ressources partagées, de files de traitement ou de mécanismes de régulation de charge.

Dans ce contexte, les performances perçues par les utilisateurs dépendent fortement de la manière dont les requêtes sont admises, mises en attente, traitées ou éventuellement rejetées. L'analyse du comportement de tels systèmes constitue donc un enjeu majeur, tant du point de vue de l'efficacité globale que de l'expérience utilisateur.

Objectifs du projet

Ce projet vise à étudier, par simulation, le comportement de plusieurs systèmes d'attente représentatifs de problématiques réelles. L'objectif n'est pas uniquement de proposer une implémentation fonctionnelle, mais surtout d'analyser l'impact des paramètres du système sur ses performances et sa stabilité.

À travers différents scénarios, il s'agit de comprendre comment les choix de conception influencent des indicateurs clés comme les temps de séjour, les taux de blocage ou la charge effective supportée par le système. L'analyse porte ainsi sur le comportement dynamique des systèmes, et non uniquement sur leur structure ou leur implémentation logicielle.

Organisation du rapport

Pour répondre à ces objectifs, une démarche fondée sur la simulation événementielle discrète a été adoptée. Chaque système étudié est modélisé à l'aide de processus stochastiques pour les arrivées et les durées de service, et plusieurs configurations de paramètres sont comparées afin d'identifier différents régimes de fonctionnement.

Les performances sont évaluées à l'aide de métriques standard issues de la théorie des files d'attente et les conclusions reposent sur une démarche de benchmarking s'appuyant sur des statistiques agrégées obtenues à partir de plusieurs exécutions indépendantes.

Le rapport est structuré autour de deux études de cas complémentaires : un premier cas de type **Waterfall**, représentant une chaîne de traitement séquentielle, puis un second cas, **Channels and Dams**, introduisant plusieurs classes d'agents et des mécanismes de régulation de la charge. Chaque étude de cas comprend la description du modèle, l'analyse de son comportement, l'évaluation par métriques et une synthèse des résultats accompagnée de recommandations.

2. Méthodologie et cadre expérimental

2.1 Principe général de simulation

Les systèmes étudiés dans ce projet sont analysés à l'aide de la simulation événementielle discrète, une méthode couramment utilisée pour l'étude des systèmes d'attente. Le fonctionnement du système est représenté comme une succession d'événements discrets, tels que l'arrivée d'une requête, le début ou la fin d'un traitement, ou encore le passage d'une étape à une autre.

Dans les différents scénarios, les arrivées des agents sont modélisées de manière aléatoire, à l'aide de lois exponentielles, ce qui correspond à des processus de Poisson classiques. Les durées de service sont également modélisées par des lois exponentielles, afin de représenter la variabilité des temps de traitement observée dans des systèmes réels.

Les ressources du système, telles que les serveurs de traitement, disposent d'une capacité limitée. Lorsque cette capacité est atteinte, les agents peuvent être mis en attente dans une file associée ou être bloqués, selon les mécanismes définis dans le modèle. Ces choix permettent d'étudier l'impact de la charge et des politiques de gestion sur les performances du système.

Chaque simulation est exécutée sur une durée fixée, suffisante pour observer le comportement global du système. Les indicateurs de performance, comme les temps de séjour ou le nombre de requêtes traitées, sont calculés à partir des événements simulés et servent de base à l'analyse présentée dans la suite du rapport.

2.2 Démarche de benchmarking et reproductibilité

Les résultats présentés dans ce rapport ne reposent pas sur une seule exécution des simulations. Pour chaque configuration étudiée, plusieurs simulations ont été réalisées avec des graines aléatoires différentes. Cela permet de prendre en compte la variabilité liée aux phénomènes aléatoires et d'éviter de tirer des conclusions à partir d'un cas particulier.

Pour chaque exécution, les métriques de performance sont calculées, puis regroupées afin d'obtenir des valeurs représentatives, comme des moyennes ou des quantiles. Les comparaisons entre différentes configurations s'appuient donc sur ces résultats agrégés, ce qui rend l'analyse plus fiable.

Lorsqu'un paramètre est étudié, les autres paramètres du système sont maintenus constants afin de mieux observer son influence sur le comportement du système. Cette méthode permet de comparer les différentes solutions de manière cohérente et d'identifier l'impact des choix de conception.

Les données issues des simulations sont conservées afin de pouvoir justifier les résultats présentés et assurer la reproductibilité des expériences.

2.3 Métriques observées

L'évaluation des systèmes d'attente étudiés dans ce projet repose sur plusieurs métriques classiques, utilisées pour analyser leurs performances et leur comportement sous charge. Les principales métriques considérées sont les suivantes :

- Temps de séjour :
temps écoulé entre l'arrivée d'un agent dans le système et la fin de son traitement.
Cette métrique est étudiée à travers :
 - sa moyenne, afin d'évaluer la latence globale du système,
 - le 95^e percentile (p95), afin de prendre en compte les situations de latence élevée.

- Nombre d'agents traités :
nombre total de requêtes complétées sur la durée de la simulation. Cette métrique permet d'évaluer la capacité effective du système à absorber la charge.
- Taux de blocage ou de rejet :
proportion de requêtes qui ne peuvent pas être admises ou traitées par le système en raison de limitations de capacité ou de mécanismes de régulation. Cette métrique est directement liée à la qualité de service perçue par l'utilisateur.
- Variance du temps de séjour :
mesure de la dispersion des temps de séjour autour de la moyenne. Une variance élevée peut traduire une expérience utilisateur inégale, même lorsque la latence moyenne reste acceptable.

Ces métriques sont utilisées de manière cohérente dans l'ensemble des études de cas afin de comparer les différentes configurations et d'identifier les compromis entre performance, stabilité et expérience utilisateur.

3. Étude de cas n°1 : *Waterfall*

3.1 Description du système

Le système *Waterfall* modélise une chaîne de traitement séquentielle, dans laquelle les requêtes traversent plusieurs étapes successives avant d'être complètement traitées. Ce type de modèle permet d'étudier l'impact des goulots d'étranglement et des capacités intermédiaires sur les performances globales d'un système d'attente.

Dans ce modèle, les requêtes arrivent dans le système selon un processus aléatoire et sont d'abord prises en charge par une étape de traitement principale, appelée la moulinette. Cette étape est modélisée par une ressource disposant d'un nombre fini de serveurs, ce qui limite le nombre de requêtes pouvant être traitées en parallèle. Lorsque tous les serveurs sont occupés, les requêtes doivent attendre qu'une ressource se libère.

Une fois le traitement principal terminé, les requêtes sont transférées vers une étape de renvoi des résultats. Cette étape est modélisée par une file d'attente associée à un serveur d'envoi unique, représentant le délai nécessaire avant que le résultat ne soit effectivement transmis à l'utilisateur. Cette seconde étape peut également devenir un point de congestion lorsque le débit d'envoi est insuffisant.

Selon la configuration étudiée, des limitations de capacité peuvent être introduites à différents niveaux du système. Lorsque ces capacités sont atteintes, certaines requêtes peuvent être bloquées, rejetées ou mises en attente via des mécanismes alternatifs, comme le stockage temporaire ou le backup. Ces choix permettent d'étudier différentes stratégies de gestion de la surcharge et leurs conséquences sur les performances et l'expérience utilisateur.

Le modèle *Waterfall* permet ainsi d'analyser le comportement d'un système soumis à une charge variable, en mettant en évidence les effets cumulés des délais à chaque étape de la chaîne de traitement.

3.2 Paramètres du modèle

Le modèle *Waterfall* est défini par un ensemble de paramètres permettant de contrôler les arrivées des requêtes, les capacités de traitement et les mécanismes de gestion de la surcharge. Ces paramètres sont fournis au simulateur sous forme d'arguments en ligne de commande et sont regroupés ci-dessous par catégorie.

Intensités et capacités de traitement

- --lam : taux d'arrivée des requêtes dans le système.
Les arrivées suivent un processus de Poisson, ce paramètre contrôle donc la charge globale imposée au système.
- --mu-exec : taux de service des serveurs d'exécution à l'étape 1.
L'inverse de ce paramètre correspond au temps moyen de traitement d'une requête par un serveur.
- --mu-send : taux de service du serveur d'envoi des résultats à l'étape 2.
Il contrôle la vitesse à laquelle les résultats sont effectivement transmis aux utilisateurs.
- --K : nombre de serveurs de traitement en parallèle à l'étape d'exécution.
Ce paramètre représente la capacité maximale de traitement simultané du système.

Capacités des files d'attente

- --ks : capacité totale du système à l'étape 1, incluant les requêtes en cours de traitement et celles en attente.
Une valeur nulle correspond à une capacité infinie. Lorsque cette capacité est atteinte, les nouvelles requêtes sont refusées à la soumission.
- --kf : capacité de la file d'attente de l'étape 2 (envoi des résultats).
Une valeur nulle indique également une capacité infinie. Lorsque cette file est pleine, le comportement dépend de la politique de gestion définie.

Politique de gestion de la surcharge (back-up)

Lorsque la file d'envoi est pleine, différentes stratégies peuvent être appliquées :

- --backup none : aucune sauvegarde n'est effectuée ; la requête est perdue, ce qui correspond à une "page blanche" côté utilisateur.
- --backup systematic : les requêtes sont systématiquement sauvegardées et réessayées ultérieurement.
- --backup probabilistic : les requêtes sont sauvegardées avec une probabilité donnée.
- --backup-p : probabilité de sauvegarde dans le cas d'une politique probabiliste.
- --backup-retry-dt : intervalle de temps entre deux tentatives de réinsertion des requêtes sauvegardées dans la file d'envoi.

Paramètres de simulation

- --sim-time : durée de génération des arrivées dans la simulation.
- --grace : période supplémentaire permettant de terminer le traitement et l'envoi des requêtes restantes après la fin des arrivées.

- --runs : nombre d'exécutions indépendantes de la simulation, utilisé pour obtenir des résultats moyennés.
- --seed : graine aléatoire permettant d'assurer la reproductibilité des expériences.

L'ensemble de ces paramètres permet d'explorer différents régimes de fonctionnement du système *Waterfall*, en faisant varier la charge, les capacités et les politiques de gestion de la surcharge, afin d'en analyser l'impact sur les performances et la stabilité du système.

3.3 Comportement du système et stabilité

Cette section analyse le comportement du système *Waterfall* en fonction des paramètres définis précédemment, en particulier la charge d'entrée, les capacités des différentes étapes et les mécanismes de gestion de la surcharge. L'objectif est d'identifier les conditions de stabilité du système ainsi que les causes de dégradation des performances.

Lorsque le taux d'arrivée des requêtes reste modéré par rapport aux capacités de traitement, le système fonctionne de manière stable. Les requêtes traversent successivement les deux étapes de la chaîne, et les temps de séjour restent limités. Dans ce régime, ni l'étape d'exécution ni l'étape d'envoi ne constituent un goulot d'étranglement majeur.

En revanche, lorsque la charge augmente, des phénomènes de congestion apparaissent. Si le nombre de serveurs à l'étape d'exécution est insuffisant ou si le taux d'arrivée devient trop élevé, les requêtes s'accumulent dans la file d'attente de la première étape. Cette accumulation entraîne une augmentation progressive des temps de séjour, pouvant conduire à un régime instable si aucune limitation n'est appliquée.

Un comportement similaire est observé lorsque l'étape d'envoi devient le goulot d'étranglement du système. Une valeur trop faible du paramètre μ -send ou une capacité limitée de la file d'envoi (k_f) peut provoquer un blocage en sortie de la chaîne. Dans ce cas, même si l'étape d'exécution fonctionne correctement, les requêtes ne peuvent pas être délivrées à un rythme suffisant, ce qui dégrade les performances globales.

L'introduction de capacités finies permet de limiter l'accumulation des requêtes, mais elle déplace le problème vers des mécanismes de blocage ou de perte. Lorsque la capacité du système d'exécution (k_s) est atteinte, les nouvelles requêtes sont refusées à la soumission. De même, lorsque la file d'envoi est pleine, certaines requêtes peuvent être perdues, ce qui correspond à des "pages blanches" du point de vue de l'utilisateur.

Les politiques de back-up permettent d'éviter ces pertes en stockant temporairement les requêtes qui ne peuvent pas être envoyées immédiatement. Ce mécanisme réduit le nombre de pages blanches, mais peut entraîner une augmentation du backlog et des temps de séjour. Le système reste alors stable au sens où il ne perd pas de requêtes, mais au prix d'une dégradation de la latence.

Ainsi, le comportement du modèle *Waterfall* met en évidence un compromis entre stabilité, latence et pertes. La stabilité du système peut être assurée soit en acceptant des refus ou des pertes, soit en conservant les requêtes au prix de délais plus importants, ce qui a un impact direct sur l'expérience utilisateur.

3.4 Évaluation par métriques

Cette section évalue les performances du modèle *Waterfall* à l'aide des métriques définies précédemment. L'analyse s'appuie sur plusieurs graphiques issus des simulations, permettant d'illustrer l'impact des paramètres de capacité et des politiques de gestion de la surcharge sur le comportement du système.

Blocage à la soumission (effet de la capacité k_s)

La Figure n°1 présente le taux de refus à la soumission en fonction de la capacité du système d'exécution (k_s).

On observe que, lorsque k_s est faible, une part importante des requêtes est refusée dès l'entrée dans le système. À mesure que k_s augmente, le taux de refus diminue rapidement, puis devient marginal au-delà d'un certain seuil.

Ce graphique montre que la capacité k_s contrôle principalement les erreurs immédiates côté utilisateur. Un sous-dimensionnement entraîne de nombreux refus, tandis qu'un sur-dimensionnement n'apporte que peu de bénéfices supplémentaires.

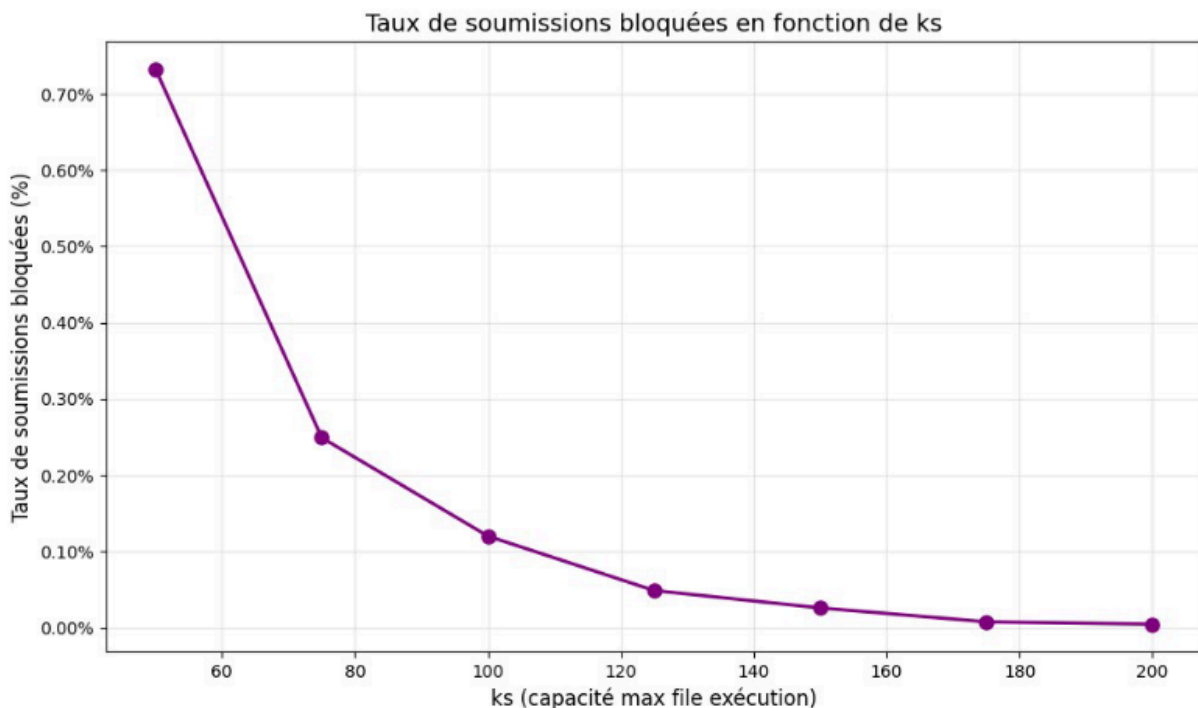


Figure n°1 : Taux de refus à la soumission en fonction de k_s

Pages blanches et capacité de la file d'envoi (k_f)

La Figure n°2 illustre le taux de pages blanches en fonction de la capacité de la file d'envoi (k_f).

Le taux de pertes diminue fortement lorsque k_f augmente, mais cette diminution n'est pas linéaire. Les gains sont importants pour des valeurs faibles de k_f , puis deviennent progressivement négligeables au-delà d'un certain seuil, situé autour de $k_f \approx 200$.

Ce comportement met en évidence des rendements décroissants : une capacité suffisante permet d'absorber les variations de charge, mais un surdimensionnement excessif n'apporte plus d'amélioration notable en termes de qualité de service.

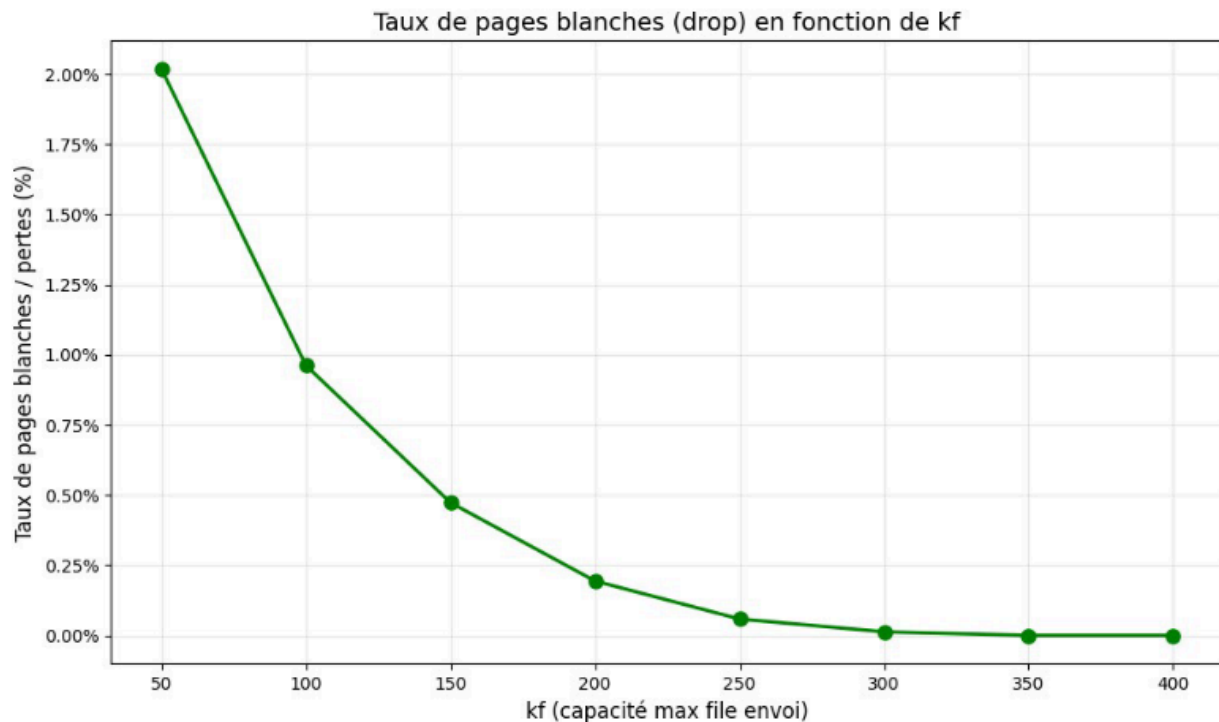


Figure n°2 : Taux de pages blanches en fonction de kf

Impact des politiques de back-up sur la latence

L'impact des politiques de back-up est analysé à l'aide de deux graphiques complémentaires.

La Figure n°3.1 présente la moyenne empirique du temps de séjour des requêtes complétées, tandis que la Figure n°3.2 montre la variance de ce temps de séjour pour les différentes politiques étudiées.

Sans back-up, les temps de séjour moyens et leur variance restent faibles pour les requêtes effectivement complétées. En revanche, cette configuration s'accompagne d'un taux élevé de pertes. À l'opposé, le back-up systématique permet de supprimer presque totalement les pages blanches, mais entraîne une augmentation très importante du temps de séjour moyen ainsi qu'une forte variance, traduisant une expérience utilisateur très dégradée et peu prévisible.

Le back-up aléatoire constitue une solution intermédiaire. Il permet de réduire une partie des pertes tout en limitant, dans une certaine mesure, l'augmentation de la latence et de sa variabilité. Ces résultats montrent que les politiques de back-up déplacent le problème : elles réduisent les pertes visibles, mais au prix d'une dégradation des délais et de la stabilité temporelle du système.

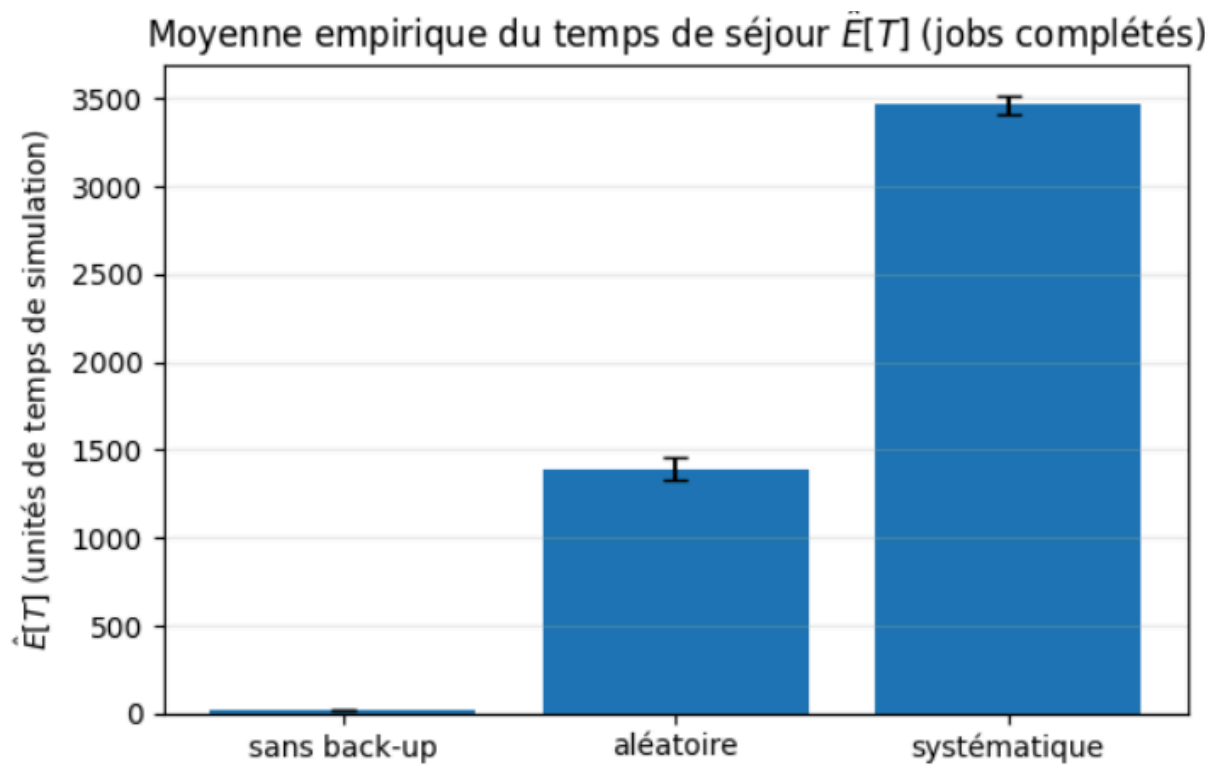


Figure n°3.1 : Moyenne du temps de séjour selon la politique de back-up

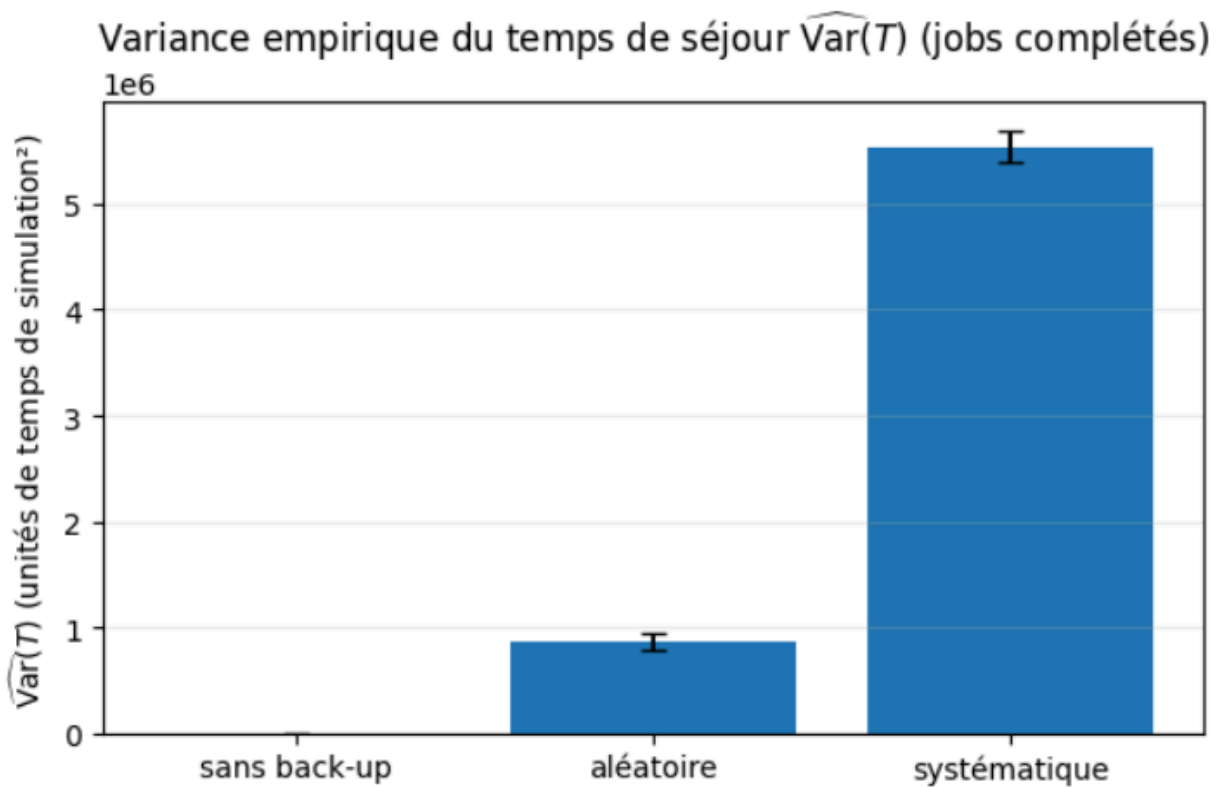


Figure n°3.2 : Variance du temps de séjour selon la politique de back-up

3.5 Synthèse et recommandations

L'étude du modèle *Waterfall* met en évidence l'impact déterminant des capacités du système et des politiques de gestion de la surcharge sur les performances et la stabilité globale. Lorsque le système n'est pas régulé ou que ses capacités sont insuffisantes, la congestion entraîne une augmentation rapide des temps de séjour et une dégradation importante de la qualité de service.

Les paramètres k_s et k_f jouent un rôle clé dans le contrôle des erreurs visibles par l'utilisateur. Un k_s trop faible provoque des refus à la soumission, tandis qu'un k_f insuffisant entraîne des pertes en sortie, correspondant à des pages blanches. Les résultats montrent qu'il existe, pour ces deux paramètres, des valeurs seuils au-delà desquelles les gains deviennent marginaux, ce qui permet de guider le dimensionnement du système.

Les politiques de back-up permettent de réduire fortement, voire d'éliminer, les pertes en sortie. Cependant, cette amélioration se fait au prix d'une augmentation significative du temps de séjour moyen et de sa variance. Le back-up systématique apparaît ainsi peu adapté à un contexte où la réactivité est prioritaire, malgré son efficacité pour éviter les pages blanches. Le back-up aléatoire offre un compromis plus acceptable, limitant partiellement les pertes tout en contenant la dégradation des délais.

Sur la base des résultats obtenus, un dimensionnement modéré des capacités (k_s et k_f) combiné à une politique de back-up non systématique apparaît comme une solution équilibrée. Ce choix permet de limiter les pertes visibles tout en évitant une explosion de la latence, offrant ainsi un compromis raisonnable entre stabilité du système et expérience utilisateur.

Enfin, il convient de souligner que ces conclusions reposent sur un modèle simplifié. En particulier, le modèle *Waterfall* ne prend pas en compte certaines contraintes externes, comme des variations temporelles de la charge ou des priorités entre requêtes, qui pourraient influencer les résultats dans un contexte réel.

4. Étude de cas n°2 : *Channels and Dams*

4.1 Description du système

Le système *Channels and Dams* modélise un système d'attente dans lequel plusieurs classes d'agents partagent une même ressource de traitement. Il vise à étudier l'impact de mécanismes de régulation de la charge sur les performances globales du système et sur l'expérience utilisateur.

Deux types d'agents sont considérés :

- les agents ING, qui génèrent des requêtes fréquentes,
- les agents PREPA, dont les arrivées sont moins fréquentes mais dont les temps de traitement peuvent être plus longs.

Les agents des deux classes partagent une ressource commune de traitement, modélisée par un nombre fini de serveurs pouvant exécuter plusieurs requêtes en parallèle. Lorsqu'un serveur est disponible, une requête peut être traitée. Dans le cas contraire, elle attend que la ressource se libère.

Après l'étape de traitement, les requêtes complétées sont transférées vers une étape de renvoi des résultats, modélisée par une file d'attente et un processus d'envoi. Cette étape permet de représenter le délai supplémentaire entre la fin du traitement et la réception effective du résultat par l'utilisateur.

Afin de contrôler la charge du système, différents mécanismes de régulation sont étudiés :

- une configuration sans régulation (*baseline*), dans laquelle toutes les requêtes sont admises,
- un mécanisme de type barrage (*dam*), qui alterne des périodes d'ouverture et de fermeture pour l'admission des requêtes ING,
- un mécanisme de type *token bucket*, dans lequel l'admission des requêtes ING dépend de la disponibilité de jetons, rechargés à un taux donné.

Dans les configurations régulées, seules les requêtes ING sont soumises à un contrôle à l'admission. Les requêtes PREPA sont toujours admises dans le système. Lorsqu'une requête ING ne peut pas être admise en raison de la régulation, elle est rejetée immédiatement et ne participe pas à la charge du système.

Ce modèle permet ainsi d'étudier les effets de différentes stratégies de régulation sur le comportement du système, en termes de temps de séjour, de taux de rejet et d'équité entre les différentes classes d'agents.

4.2 Paramètres du modèle

Le comportement du système *Channels and Dams* est contrôlé par différents paramètres définissant les flux d'entrée, les capacités de traitement et les mécanismes de régulation. Ces paramètres sont fournis au simulateur via des arguments en ligne de commande et sont présentés ci-dessous.

Populations et flux d'entrée

- `--lam-ing` : taux d'arrivée des requêtes de type ING (par exemple, 2.0 correspond à deux arrivées par unité de temps de simulation).
- `--lam-prepa` : taux d'arrivée des requêtes de type PREPA, généralement plus faible que celui des ING.
- `--mu-ing` : vitesse de traitement des requêtes ING (l'inverse de ce paramètre correspond au temps moyen de service).
- `--mu-prepa` : vitesse de traitement des requêtes PREPA, souvent plus lente que celle des ING.
- `--mu-send` : vitesse du serveur chargé de l'envoi des résultats après le traitement.

Système et régulation

- `--K` : nombre de serveurs de traitement en parallèle, représentant la capacité maximale de la moulinette.
- `--mode` : stratégie de contrôle de la charge :
 - *baseline* : aucune régulation, toutes les requêtes sont admises,

- *dam* : régulation par barrage périodique, alternant des phases de fermeture et d'ouverture de l'admission des requêtes ING,
- *token* : régulation par débit à l'aide d'un mécanisme de type *token bucket*.
- --t-b : durée pendant laquelle le barrage est bloqué (fermé) dans le mode *dam*.
- --t-o : durée pendant laquelle le barrage est ouvert dans le mode *dam*.
- --token-rate : vitesse de génération des jetons, correspondant au débit autorisé pour les requêtes ING.
- --token-cap : capacité maximale du réservoir de jetons, permettant d'absorber des rafales de requêtes.

Paramètres d'administration de la simulation

- --sim-time : durée totale de la simulation.
- --runs : nombre de répétitions de la simulation, utilisé pour obtenir des résultats moyennés et plus stables.
- --seed : graine aléatoire assurant la reproductibilité des expériences.

L'ensemble de ces paramètres permet de faire varier de manière contrôlée la charge du système et d'analyser l'impact des différents mécanismes de régulation sur les performances, la stabilité et l'expérience utilisateur.

4.3 Comportement du système et stabilité

Cette partie présente le comportement du système *Channels and Dams* selon le mécanisme de régulation utilisé : sans régulation (*baseline*), avec un barrage (*dam*), ou avec un mécanisme de type *token bucket*. L'objectif est d'analyser l'impact de ces mécanismes sur la stabilité du système et sur les performances observées.

En configuration *baseline*, toutes les requêtes ING et PREPA sont admises dans le système. Lorsque la charge reste raisonnable, le système fonctionne de manière stable. En revanche, lorsque les taux d'arrivée augmentent, les requêtes s'accumulent et les temps de séjour augmentent rapidement, ce qui peut conduire à une dégradation importante des performances.

Le mécanisme de barrage (*dam*) permet de limiter la charge en bloquant périodiquement l'admission des requêtes ING. Cette régulation améliore la stabilité du système et réduit les temps de séjour pour les requêtes traitées. En contrepartie, elle entraîne un rejet important des requêtes ING, ce qui peut nuire à l'expérience utilisateur.

Le mécanisme de *token bucket* repose sur une limitation du débit d'admission des requêtes ING. Dans le cadre de la simulation, les arrivées ING sont continues, ce qui conduit à une consommation rapide des jetons disponibles. Le paramètre *token-cap* a alors un impact limité : il permet seulement à un certain nombre de requêtes ING supplémentaires d'être admises au début de la simulation, sans modifier significativement le comportement à long terme. L'effet principal de la régulation est donc piloté par le paramètre *token-rate*.

Dans un contexte plus réaliste, avec des arrivées par vagues ou des périodes de faible activité, le paramètre *token-cap* jouerait un rôle plus important en permettant d'absorber des pics de charge. L'absence de ce type de comportement dans la simulation constitue une limite du modèle étudié.

Globalement, les mécanismes de régulation permettent de stabiliser le système en réduisant la charge admise. Le barrage offre une régulation plus stricte mais plus brutale, tandis que le *token bucket* propose un compromis plus progressif entre stabilité et taux de rejet.

4.4 Évaluation du système au regard des métriques

Nous évaluons dans cette section les performances du système *Channels and Dams* en comparant les trois mécanismes étudiés (*baseline*, *dam* et *token*) à l'aide des métriques définies précédemment. L'analyse s'appuie sur les graphiques présentés ci-dessous, qui illustrent l'impact de la régulation sur la latence, la stabilité et le taux de rejet.

Temps de séjour moyen

La figure n°4 présente le temps de séjour moyen des requêtes ING et PREPA pour les trois configurations étudiées.

On observe que, dans le cas *baseline*, les temps de séjour sont très élevés pour les deux populations, dépassant plusieurs centaines de secondes. Cela traduit un système fortement congestionné lorsque aucune régulation n'est appliquée.

À l'inverse, l'introduction de mécanismes de régulation permet de réduire fortement la latence. Le mécanisme de barrage (*dam*) conduit aux temps de séjour les plus faibles, tandis que le *token bucket* présente des temps légèrement plus élevés, mais restant très largement inférieurs à ceux observés sans régulation. Ces résultats montrent que la régulation a un impact majeur sur la latence moyenne du système.

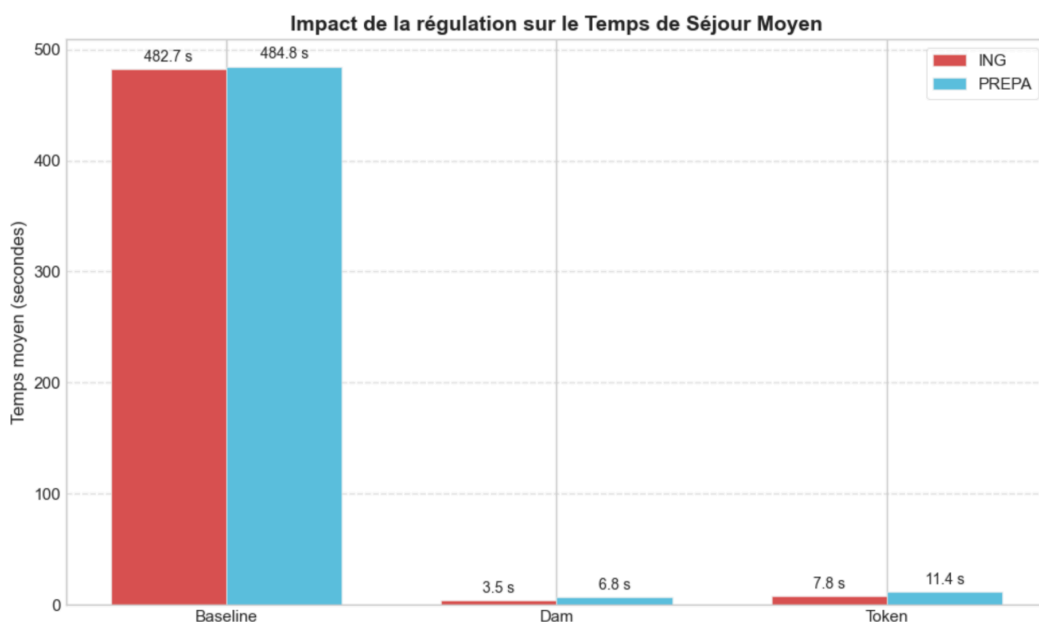


Figure n°4 : Impact de la régulation sur le temps de séjour moyen

Stabilité du système (p95 du temps de séjour)

La Figure n°5 illustre le 95^e percentile du temps de séjour pour les requêtes ING et PREPA. Cette métrique permet d'évaluer la stabilité du système et les situations de latence extrême.

Dans la configuration *baseline*, le p95 est très élevé, ce qui indique que certaines requêtes subissent des délais particulièrement longs. Les mécanismes de régulation permettent de réduire

fortement ce phénomène. Le barrage et le *token bucket* présentent des valeurs de p95 beaucoup plus faibles, traduisant un comportement plus stable et plus prévisible du système.

Ces résultats confirment que la régulation améliore non seulement la latence moyenne, mais aussi la stabilité perçue par les utilisateurs.

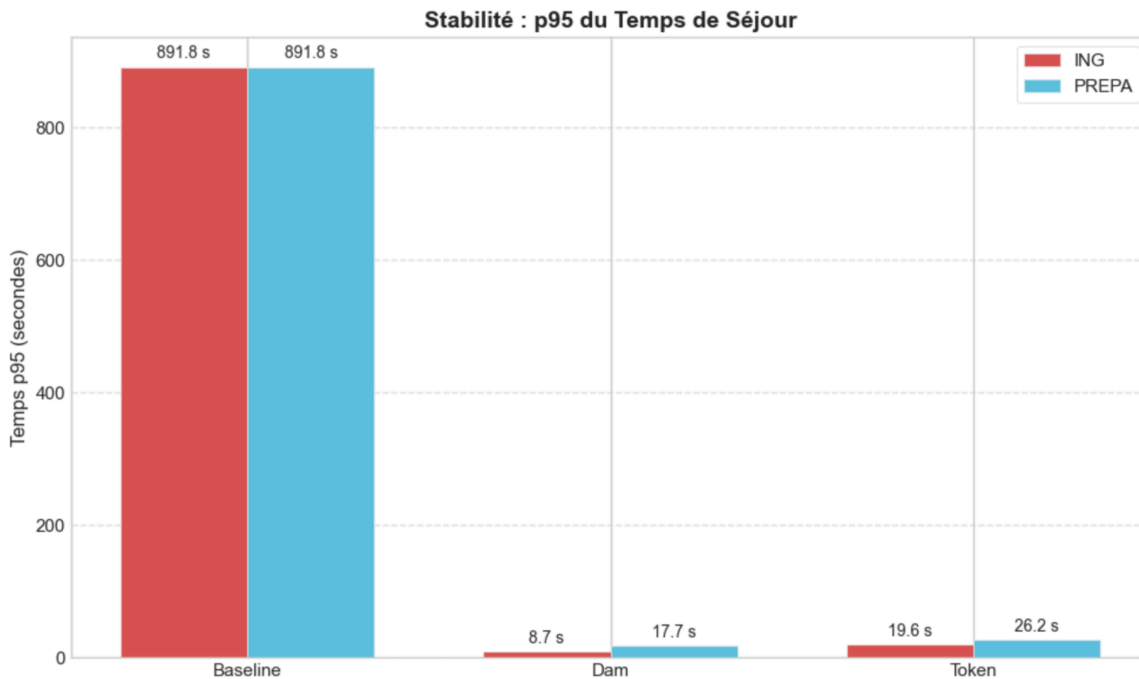


Figure n°5 : p95 du temps de séjour pour les différentes configurations

Coût de la régulation : taux de rejet des requêtes ING

La Figure n°6 présente le taux de rejet des requêtes ING pour les différentes configurations.

En configuration baseline, aucune requête n'est rejetée. En revanche, les mécanismes de régulation introduisent un coût sous forme de rejets explicites.

Le mécanisme de barrage entraîne un taux de rejet très élevé, supérieur à 60 %, ce qui correspond à une régulation stricte mais brutale.

Le token bucket présente un taux de rejet plus modéré, autour de 25 %, traduisant une régulation plus progressive.

Ce graphique met en évidence le compromis inhérent à la régulation : la réduction de la latence s'accompagne nécessairement d'un rejet d'une partie des requêtes.

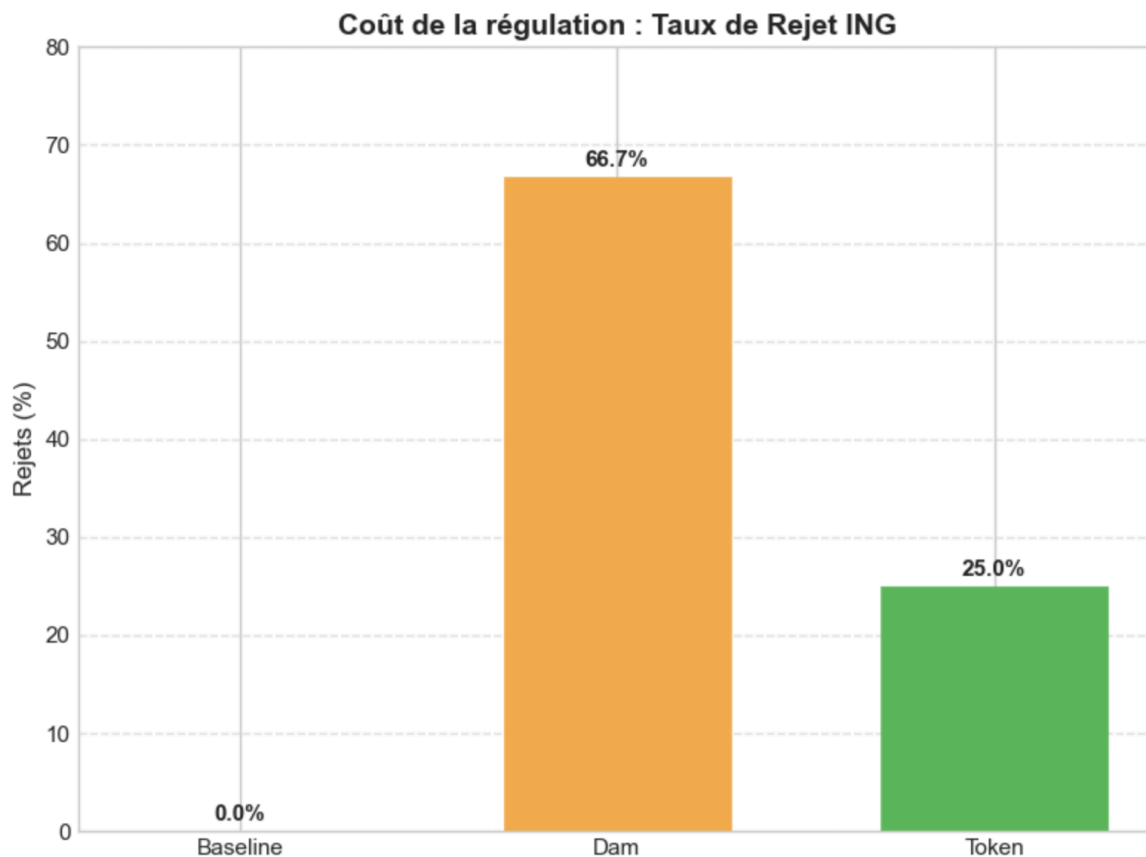


Figure n°6 : Taux de rejet des requêtes ING

Compromis entre qualité de service et rejet

La Figure n°7 synthétise les résultats précédents en représentant le compromis entre la latence moyenne et le taux de rejet.

La configuration *baseline* se caractérise par un taux de rejet nul, mais une latence très élevée. Le *dam* se situe à l'opposé, avec une latence minimale mais un taux de rejet très important. Le *token bucket* occupe une position intermédiaire, offrant un compromis plus équilibré entre qualité de service et rejet.

Ce graphique met en évidence l'absence de solution idéale et souligne l'importance du choix du mécanisme de régulation en fonction des priorités du système, notamment en matière d'expérience utilisateur.

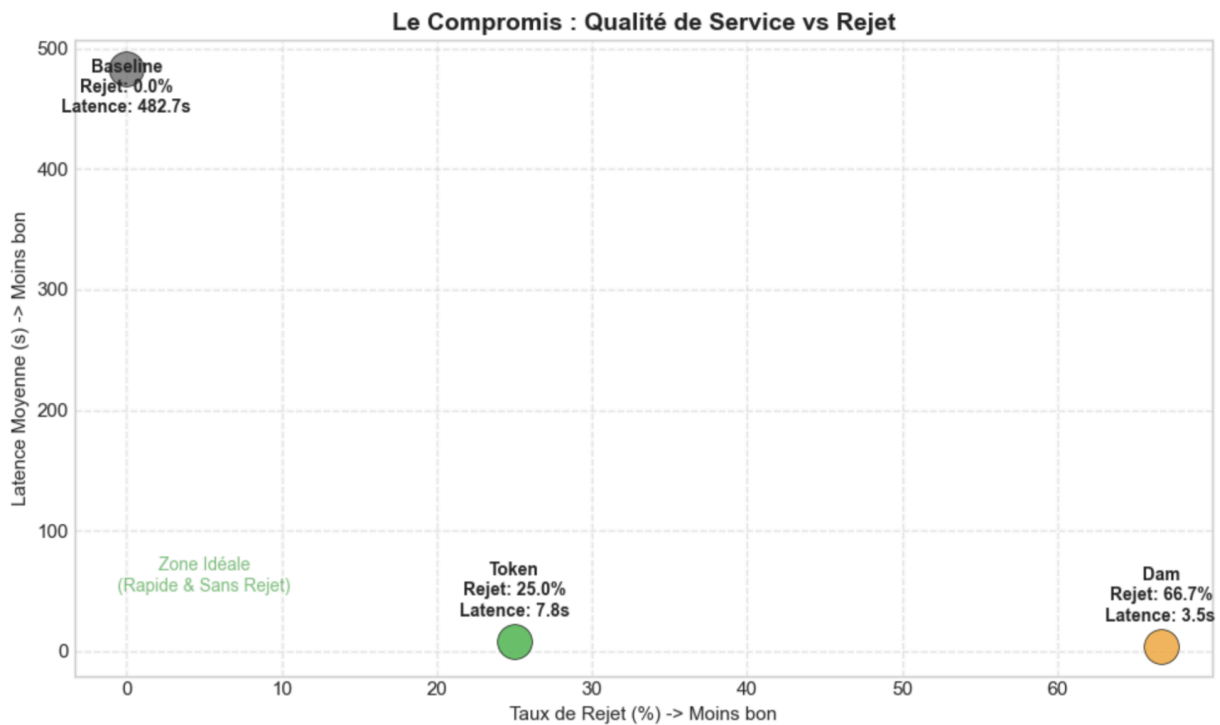


Figure n°7 : Compromis entre qualité de service et taux de rejet

4.5 Synthèse et recommandations

L'étude du système *Channels and Dams* met en évidence l'impact majeur des mécanismes de régulation sur le comportement et les performances du système. Sans régulation, le système admet toutes les requêtes, mais présente des temps de séjour très élevés et une forte instabilité, ce qui conduit à une dégradation importante de la qualité de service.

L'introduction de mécanismes de régulation permet de réduire significativement la latence et d'améliorer la stabilité du système. Le mécanisme de barrage (*dam*) est le plus efficace pour minimiser les temps de séjour, mais il entraîne un taux de rejet très élevé des requêtes ING. Cette solution peut être adaptée dans des contextes où la priorité est donnée à la rapidité de traitement, mais elle présente un risque important pour l'expérience utilisateur en raison du nombre élevé de rejets.

Le mécanisme de *token bucket* propose un compromis plus équilibré. Il permet de réduire fortement les temps de séjour tout en limitant le taux de rejet à un niveau plus modéré. Cette approche apparaît plus adaptée à un contexte réel, où il est nécessaire de concilier performance du système et acceptabilité côté utilisateur.

Sur la base des résultats obtenus, le *token bucket* constitue la solution la plus pertinente parmi celles étudiées pour assurer un comportement stable et efficace du système, tout en maintenant un niveau de rejet raisonnable. Toutefois, le choix du mécanisme de régulation doit rester dépendant des contraintes du système et des priorités en matière de qualité de service.

Enfin, il convient de rappeler que les résultats présentés reposent sur un modèle de simulation simplifié. En particulier, l'absence de variations temporelles dans les flux d'arrivée limite l'impact de certains paramètres, comme la capacité du réservoir de jetons.

5. Discussion transversale et limites

Les deux études de cas présentées dans ce rapport abordent des problématiques communes de gestion de charge et de systèmes d'attente, mais reposent sur des architectures et des stratégies de contrôle fondamentalement différentes. Le modèle *Waterfall* s'appuie sur une chaîne de traitement séquentielle, tandis que *Channels and Dams* introduit un contrôle explicite de l'admission des requêtes. Cette différence structurelle conduit à des comportements distincts face à la surcharge.

Dans le modèle *Waterfall*, la gestion de la charge est principalement assurée par le dimensionnement des capacités internes et par des mécanismes de gestion de la surcharge en sortie, comme le back-up. Lorsque la charge augmente, le système tend à accumuler des requêtes, ce qui se traduit par une augmentation des temps de séjour et, en l'absence de mécanisme adapté, par des pertes visibles pour l'utilisateur (pages blanches). La stabilité peut être préservée soit en acceptant des refus ou des pertes, soit en conservant les requêtes au prix d'une latence accrue.

À l'inverse, le modèle *Channels and Dams* agit plus en amont en contrôlant l'admission des requêtes. Les mécanismes de barrage et de *token bucket* permettent de limiter la charge admise dans le système, ce qui améliore fortement la stabilité et réduit les temps de séjour. Cependant, cette approche introduit un coût explicite sous forme de rejets, qui impacte directement l'expérience utilisateur. Le *token bucket* apparaît comme une solution intermédiaire, offrant un meilleur compromis entre performance et taux de rejet.

La comparaison des deux modèles met ainsi en évidence deux philosophies différentes. Le modèle *Waterfall* privilégie l'admission des requêtes et tente de gérer la surcharge à l'intérieur du système, tandis que *Channels and Dams* privilégie la stabilité en filtrant la charge dès l'entrée. Dans le premier cas, l'utilisateur est davantage exposé à des délais élevés ou à des erreurs tardives. Dans le second, il peut être confronté à des rejets immédiats mais bénéficie de délais plus courts pour les requêtes acceptées.

Ces résultats doivent toutefois être interprétés à la lumière des limites des modèles étudiés. Les simulations reposent sur des hypothèses simplificatrices, notamment des arrivées modélisées par des processus de Poisson et des durées de service exponentielles. De plus, certaines situations réalistes ne sont pas prises en compte, comme des variations temporelles marquées de la charge ou des comportements utilisateurs adaptatifs. Par exemple, dans *Channels and Dams*, l'absence d'arrivées par vagues limite l'impact du paramètre de capacité du *token bucket*, alors que celui-ci pourrait jouer un rôle plus important dans un contexte réel.

Enfin, les métriques utilisées se concentrent principalement sur la latence, les rejets et les pertes. D'autres critères, tels que l'équité entre populations ou la priorisation dynamique des requêtes, pourraient enrichir l'analyse et modifier les conclusions.

6. Conclusion et recommandations générales

Ce projet a permis d'analyser le comportement de différents systèmes d'attente à l'aide de la simulation événementielle discrète. À travers deux études de cas complémentaires, *Waterfall* et *Channels and Dams*, nous avons étudié l'impact des paramètres de charge, de capacité et des mécanismes de régulation sur les performances, la stabilité et l'expérience utilisateur.

L'étude du modèle *Waterfall* a mis en évidence l'importance du dimensionnement des capacités internes et des politiques de gestion de la surcharge. Les résultats montrent que l'augmentation des capacités permet de réduire les refus et les pertes jusqu'à un certain seuil, au-delà duquel les gains deviennent marginaux. Les mécanismes de back-up permettent d'éviter les pertes visibles, mais au prix d'une augmentation significative des temps de séjour et de leur variabilité, ce qui peut fortement dégrader la qualité de service perçue.

Le modèle *Channels and Dams* propose une approche différente, fondée sur un contrôle explicite de l'admission des requêtes. Les mécanismes de *dams* et de *token bucket* permettent de stabiliser efficacement le système et de réduire les temps de séjour. Toutefois, cette stabilité s'accompagne d'un rejet d'une partie des requêtes, ce qui constitue un coût direct pour l'utilisateur. Parmi les solutions étudiées, le *token bucket* apparaît comme le meilleur compromis entre performance et acceptabilité, en limitant à la fois la latence et le taux de rejet.

La comparaison des deux modèles met en évidence un compromis fondamental entre latence, stabilité et rejet. Il n'existe pas de solution universelle : le choix de l'architecture et des mécanismes de régulation dépend des priorités du système et des contraintes liées à l'expérience utilisateur. Dans un contexte réel, une combinaison de ces approches pourrait être envisagée, par exemple en associant un contrôle à l'admission à une gestion fine des capacités internes.

Enfin, ce travail présente certaines limites liées aux hypothèses de modélisation retenues, notamment l'absence de variations temporelles de la charge ou de comportements utilisateurs adaptatifs. Des extensions futures pourraient intégrer ces aspects afin d'obtenir une analyse plus proche de situations réelles et d'affiner les recommandations proposées.

Annexes

Ces annexes regroupent les figures présentées dans le corps du rapport dans un format plus lisible. Les figures sont reprises sans modification des données, uniquement avec un agrandissement et, le cas échéant, une meilleure résolution afin de faciliter leur lecture.

Figure n°1 : Taux de refus à la soumission en fonction de ks.....	19
Figure n°3.1 : Moyenne du temps de séjour selon la politique de back-up.....	20
Figure n°3.2 : Variance du temps de séjour selon la politique de back-up.....	20
Figure n°4 : Impact de la régulation sur le temps de séjour moyen.....	21
Figure n°5 : p95 du temps de séjour pour les différentes configurations.....	21
Figure n°6 : Taux de rejet des requêtes ING.....	22
Figure n°7 : Compromis entre qualité de service et taux de rejet.....	22

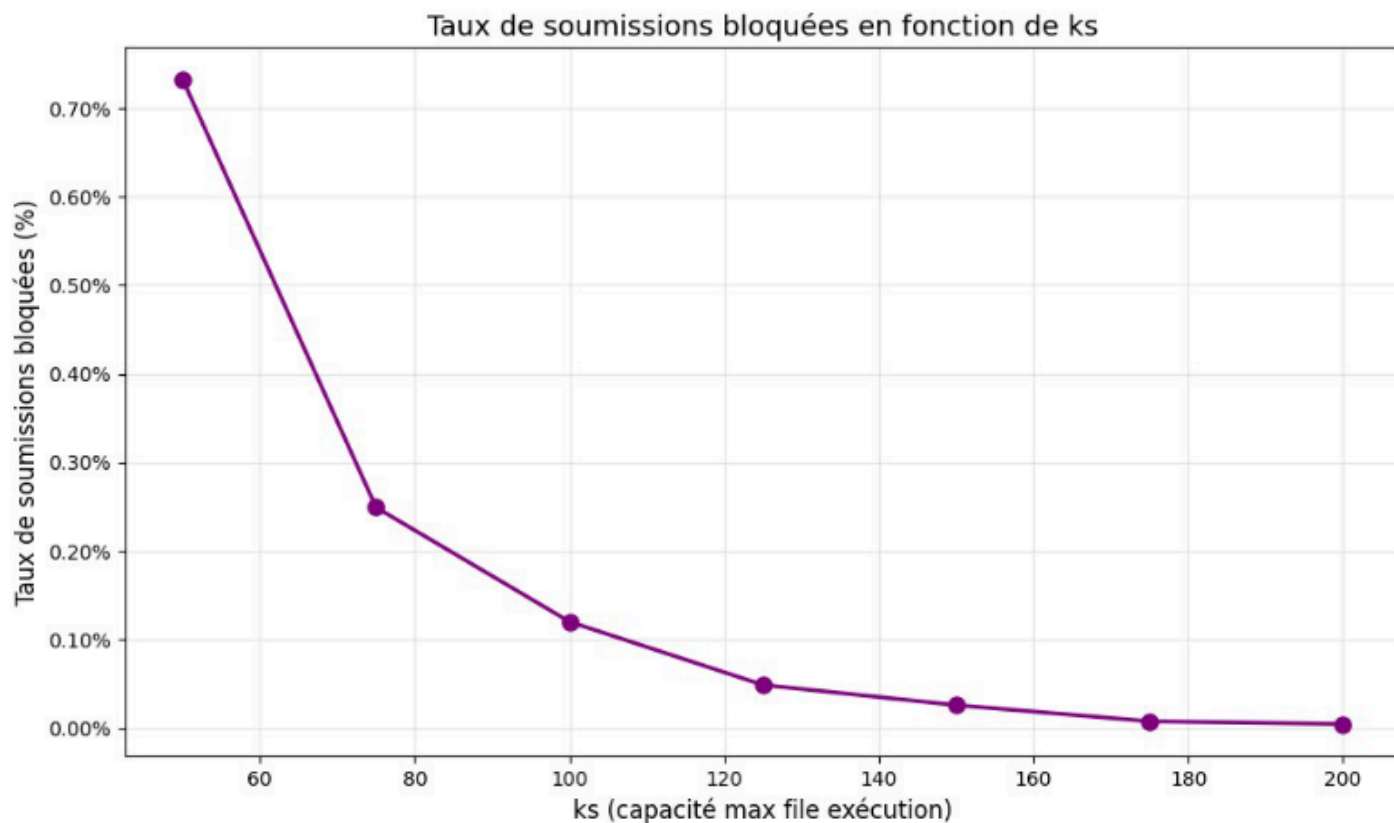
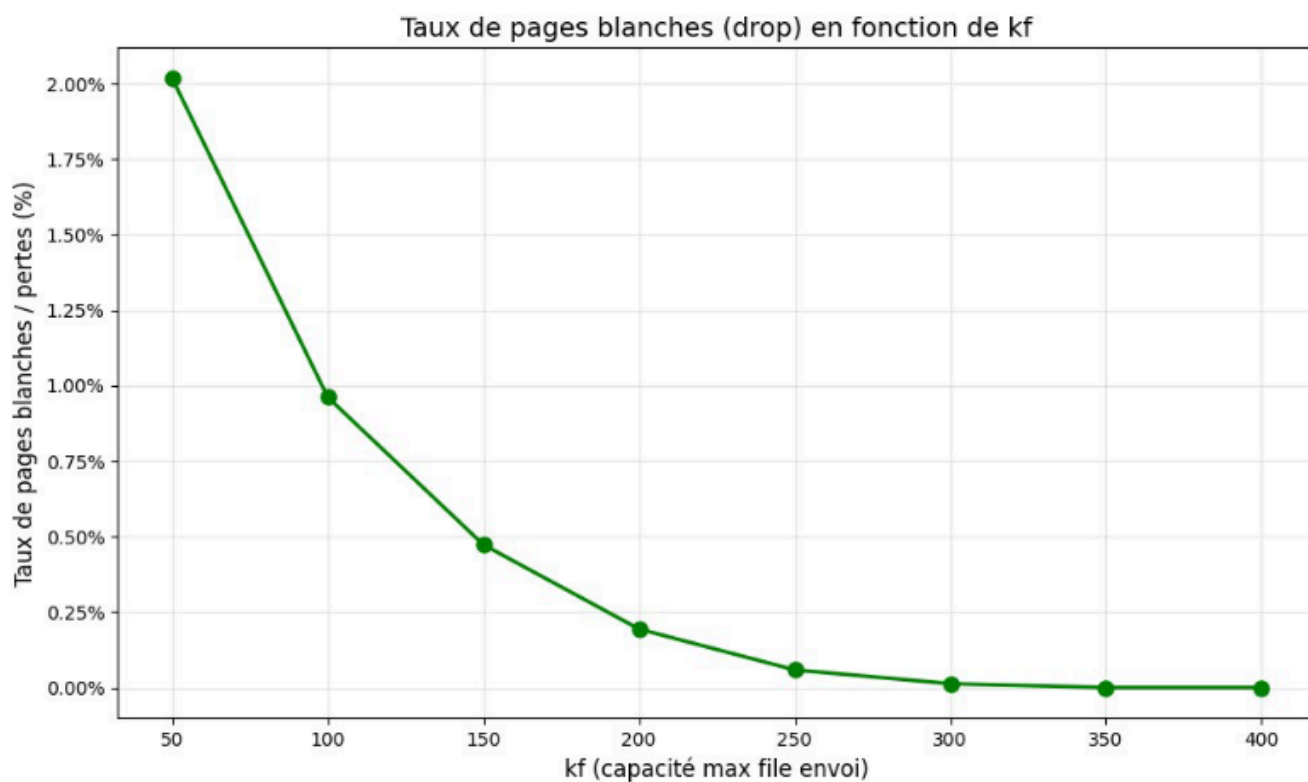


Figure n°1 : Taux de refus à la soumission en fonction de ks



.Figure n°2 : Taux de pages blanches en fonction de kf

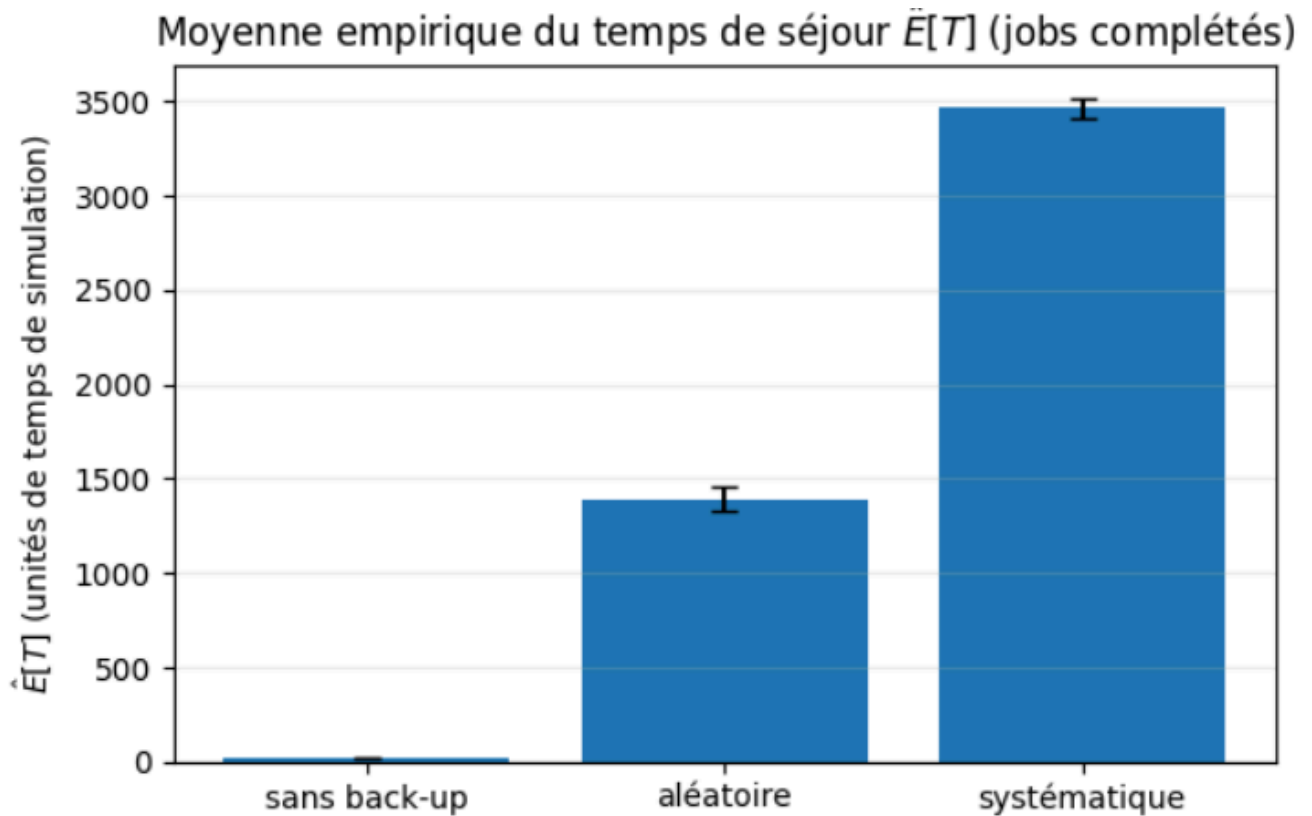


Figure n°3.1 : Moyenne du temps de séjour selon la politique de back-up

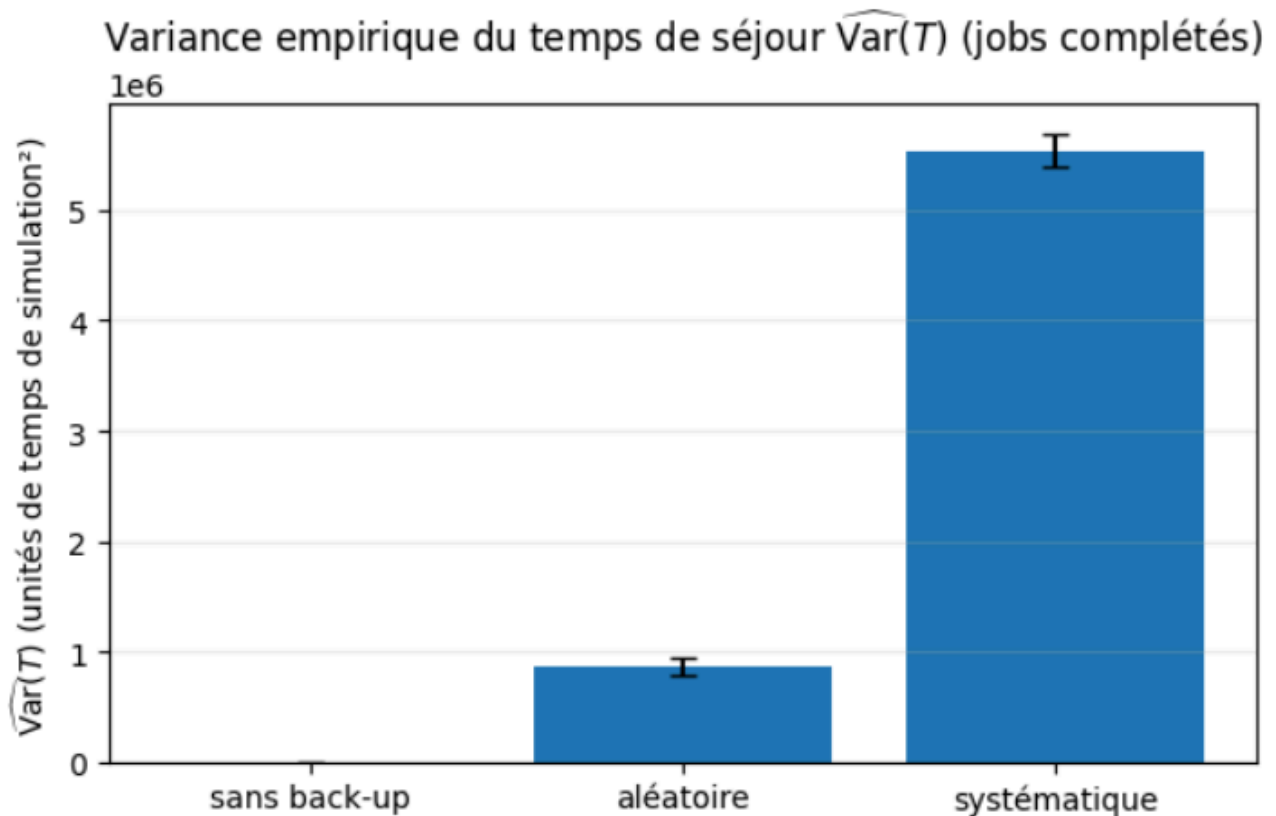


Figure n°3.2 : Variance du temps de séjour selon la politique de back-up

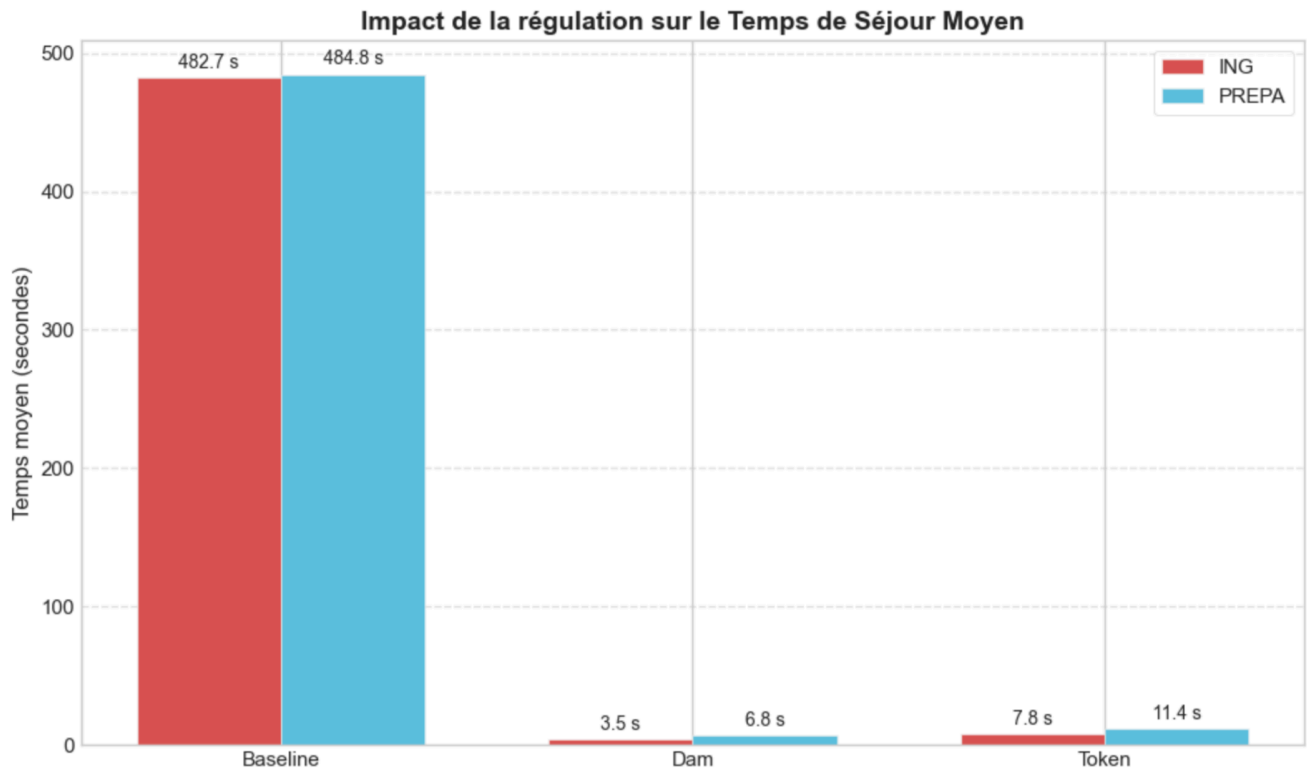


Figure n°4 : Impact de la régulation sur le temps de séjour moyen

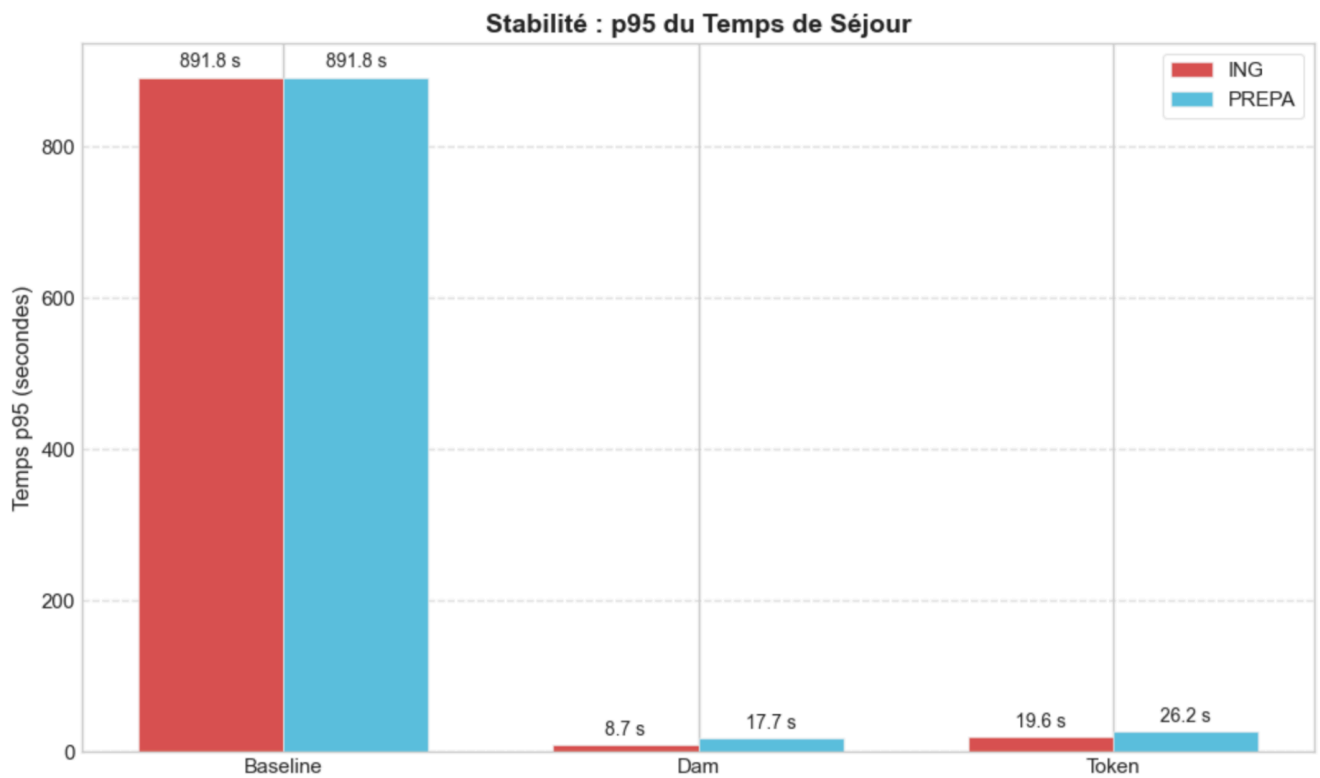


Figure n°5 : p95 du temps de séjour pour les différentes configurations

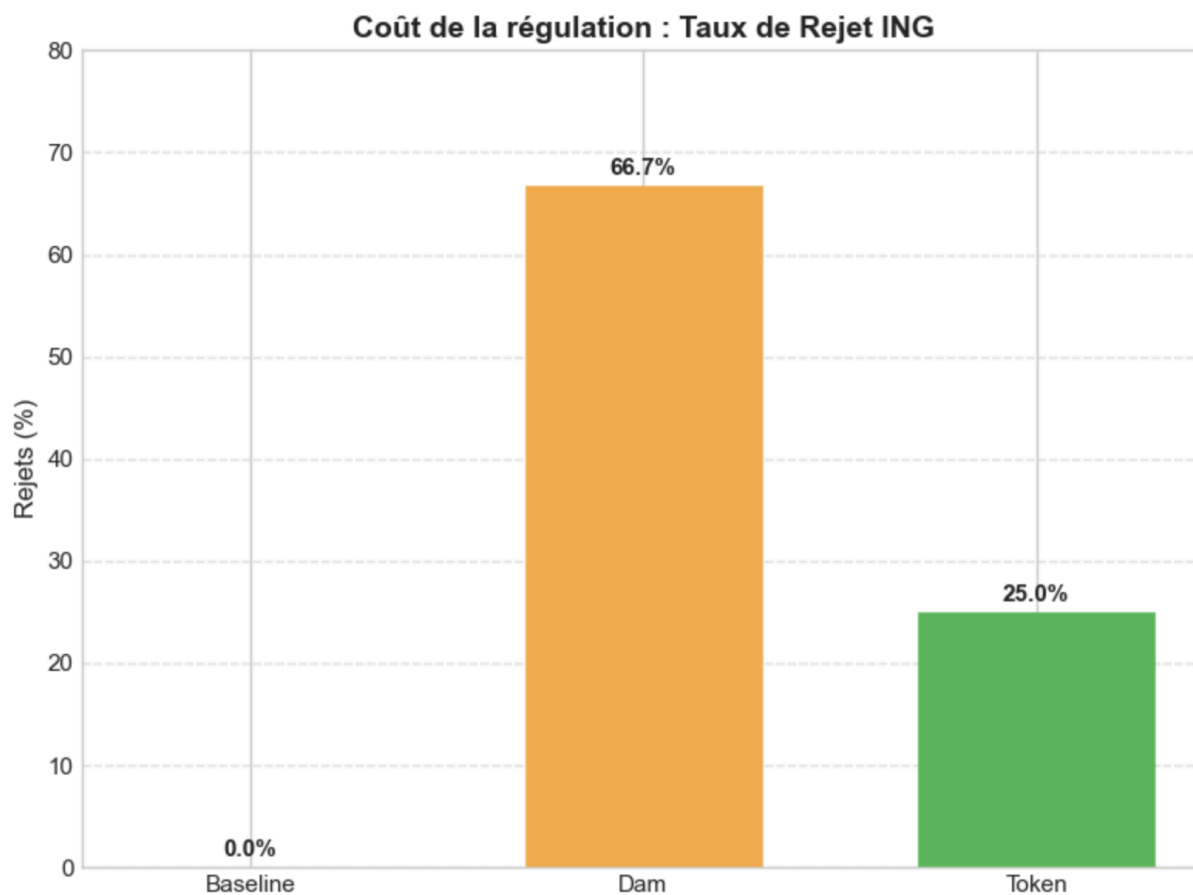


Figure n°6 : Taux de rejet des requêtes ING

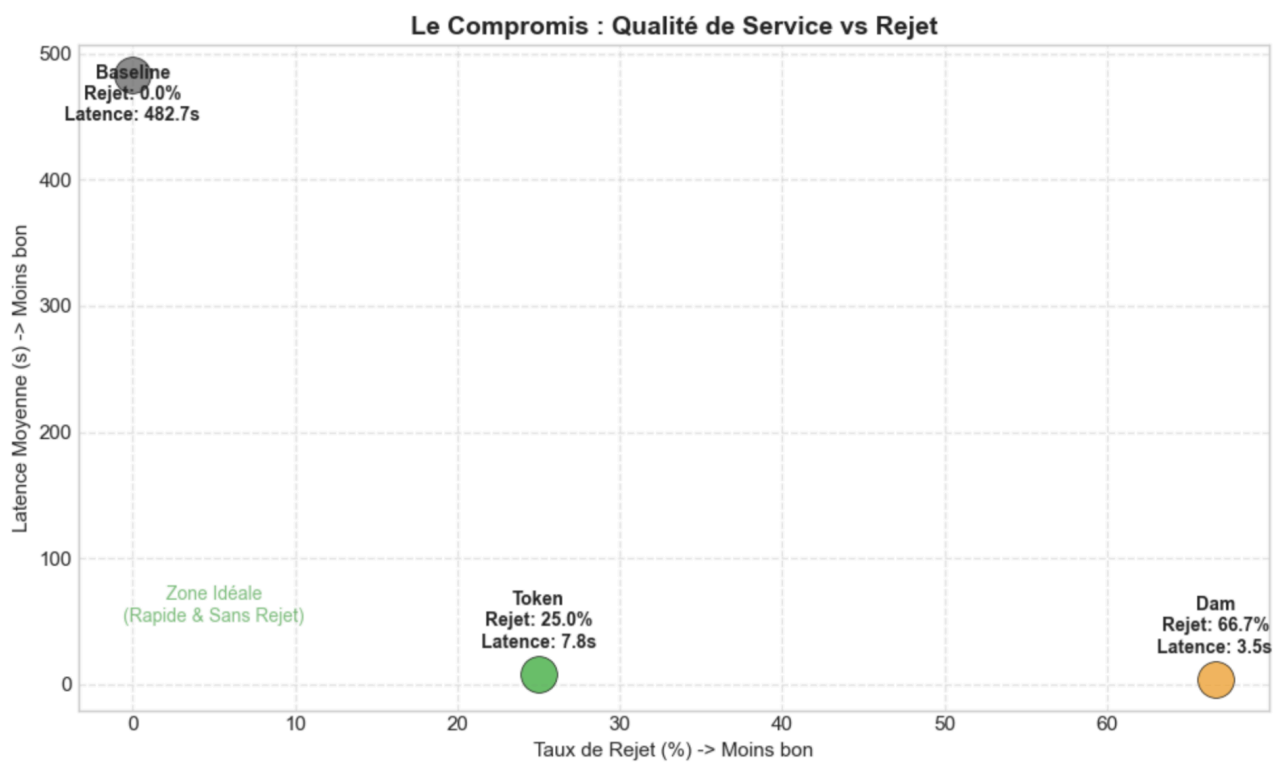


Figure n°7 : Compromis entre qualité de service et taux de rejet